

Homotopy Type Theory Meets Judgment Geometry

Halley Young
Microsoft Research
halleyyoung@microsoft.com

March 22, 2026

Abstract

We develop a homotopy-type-theoretic interpretation of the Judgment Geometry framework. Judgments are recast as *types* in Martin-Löf type theory with univalence (MLTT +UA); a verified program is a *term* inhabiting the type $\Gamma(\mathcal{S}, \mathbf{JType})$ — the global sections of the judgment-type sheaf. The ∞ -topos $\mathbf{Sh}(\mathcal{S})$ of sheaves on the semantic site $\mathcal{S}$ provides a model of MLTT in which types are sheaves, terms are sections, and the univalence axiom holds for program types. Under this interpretation, extensional program equivalence ($A \approx B$) corresponds to a *path* in $\mathbf{Prog}$, and the univalence axiom promotes every equivalence to a propositional equality. We show that the fundamental group $\pi_1(\mathbf{Prog}, A)$ classifies *refactoring loops* — verified refactoring sequences that return A to itself up to homotopy. As a principal application, we derive an automated *proof-transfer* principle: any property $P: \mathbf{Prog} \rightarrow \mathcal{U}$ proved for program A transfers automatically to any B that is path-connected to A (i.e., obtainable from A by a sequence of verified refactorings), with the transferred proof constructed by transport along the connecting path. The theory is validated against a companion LEAN 4.4 development in which all central theorems are proved without `sorry`.

Contents

1	Introduction	2
2	Background: Martin-Löf Type Theory and HoTT	2
2.1	Universes and the Univalence Axiom	3
2.2	Identity Types and Transport	3
2.3	Higher Inductive Types	3
3	Semantic Sites as ∞-Categories	3
3.1	The Semantic Site of JuGeo	4
4	The ∞-Topos $\mathbf{Sh}(\mathcal{S})$ as a Model of HoTT	4
4.1	Types as Sheaves and Terms as Sections	5
5	Judgment Types and Evidence Bundles	5
6	Univalence for Program Semantics	6
6.1	The Type of Programs	6
6.2	Behavioral Equivalence as Equivalence of Types	6

7	Higher Inductive Types for Program Transformations	6
7.1	The Refactoring Interval	6
7.2	The Space of Refactorings	7
7.3	Pushout Programs	7
8	Proof Transfer via Transport	7
8.1	Transfer of JU GEO Judgment Certificates	8
9	The Fundamental Group of Programs	8
9.1	Example: The Loop Group of a Pure Function	9
10	Implementation in JuGeo	9
10.1	Univalence in the Lean Model	10
11	The Čech–de Rham Spectral Sequence Revisited	10
12	Related Work	10
13	Conclusion	11

1 Introduction

Formal verification and homotopy type theory (HoTT) share a common root in the Curry–Howard correspondence: a proof is a term, and a proposition is a type. Yet the two traditions have developed largely in parallel. Proof-based verification frameworks (LEAN 4, COQ, F*) work within a fixed universe hierarchy and treat program *equality* as a synthetic notion bolted onto an underlying set-theoretic model. HoTT [1], by contrast, internalises the geometry of proof spaces: two terms $a, b : A$ may be connected by a family of *paths*, higher-dimensional witnesses of identification, and the space of all such paths carries its own algebraic structure — an ∞ -groupoid.

The Judgment Geometry series (Papers 1–19) builds program verification on the notion of a *semantic site* $\mathcal{S}$: a Grothendieck site whose objects are program fragments, whose morphisms are semantic embeddings, and whose covers encode correctness-preserving decompositions. Verified programs are *global sections* of judgment sheaves; verification failures are cohomological obstructions.

This paper closes a conceptual circle by showing that the sheaf-topos $\mathbf{Sh}(\mathcal{S})$ is not merely a model of ordinary first-order logic but a full model of MLTT with univalence. The payoff is three-fold:

1. **Paths are refactorings.** A path $p : A =_{\mathbf{Prog}} B$ in the type of programs is exactly a *verified refactoring* from A to B : a morphism in $\mathcal{S}$ that transports all judgment-sheaf sections.
2. **Univalence is behavioral equivalence.** The univalence axiom asserts $(A \simeq_{\mathbf{Prog}} B) \simeq (A =_{\mathbf{Prog}} B)$. Behaviorally equivalent programs become propositionally equal, so any type-theoretic property proved of one holds of the other.
3. **Proof transfer is transport.** Given $P : \mathbf{Prog} \rightarrow \mathcal{U}$ and a path $p : A =_{\mathbf{Prog}} B$, the function $\text{transport}(P, p) : P(A) \rightarrow P(B)$ transfers proofs along refactorings with no additional human effort.

Contributions.

- We construct a model of MLTT +UA in $\mathbf{Sh}(\mathcal{S})$ (Theorem 4.2).
- We identify program equivalence with paths in $\mathbf{Prog}$ (Theorem 6.3).
- We classify the fundamental group $\pi_1(\mathbf{Prog}, A)$ as the group of refactoring loops (Theorem 9.2).
- We formalise proof transfer and validate it in LEAN 4 4 (Section 10).

Organisation. Section 2 reviews the necessary HoTT background. Section 3 recasts semantic sites as ∞ -categories. Section 4 constructs the ∞ -topos model. Sections 5–7 develop judgment types, univalence, and higher inductive types. Sections 8–9 present proof transfer and the fundamental group. Section 10 describes the LEAN 4 4 companion. Sections 12–13 close the paper.

2 Background: Martin-Löf Type Theory and HoTT

We recall the fragment of HoTT needed for our construction; the standard reference is the HoTT Book [1].

2.1 Universes and the Univalence Axiom

MLTT is stratified into a hierarchy of universes $\mathcal{U}_0 : \mathcal{U}_1 : \mathcal{U}_2 : \dots$ where each $\mathcal{U}_i$ is a type whose terms are types of the next-lower universe. For clarity we write $\mathcal{U}$ when the level is unimportant.

Definition 2.1 (Equivalence). A function $f : A \rightarrow B$ is an *equivalence* if it has a quasi-inverse: there exists $g : B \rightarrow A$ with $\eta : \prod_{a:A} (g(f(a)) = a)$ and $\varepsilon : \prod_{b:B} (f(g(b)) = b)$. We write $A \simeq B$ for the type of equivalences from A to B .

The **Univalence Axiom** [2] asserts that for any two types $A, B : \mathcal{U}$, the canonical map

$$\text{ua} : (A \simeq B) \longrightarrow (A = B)$$

is itself an equivalence. Consequently, $(A \simeq B) \simeq (A = B)$: equivalent types are propositionally equal.

2.2 Identity Types and Transport

For $A : \mathcal{U}$ and $a, b : A$, the *identity type* $a =_A b$ is inhabited by *paths* from a to b . Paths compose (by path concatenation $\cdot$), invert, and satisfy the groupoid laws up to higher paths.

Definition 2.2 (Transport). Given $P : A \rightarrow \mathcal{U}$, a path $p : a =_A b$, and a term $u : P(a)$, the *transport* of u along p is the canonical term

$$\text{transport}(P, p, u) : P(b).$$

Transport is the fundamental dependent eliminator for identity types.

2.3 Higher Inductive Types

A *higher inductive type* (HIT) is a type defined by point constructors *and* path constructors (and possibly higher-path constructors). The canonical example is the *interval type* $\mathbb{I}$ with:

- two point constructors $0_{\mathbb{I}}, 1_{\mathbb{I}} : \mathbb{I}$;
- one path constructor $\text{seg} : 0_{\mathbb{I}} =_{\mathbb{I}} 1_{\mathbb{I}}$.

More complex HITs include the circle S^1 , suspension types, and the quotient types used in Section 7.

3 Semantic Sites as ∞ -Categories

Papers 1–3 developed semantic sites as 1-categorical Grothendieck sites. For the HoTT connection we need a mild upgrade.

Definition 3.1 (Semantic ∞ -site). A *semantic ∞ -site* is a pair $(\mathcal{S}, J)$ where

1. $\mathcal{S}$ is an ∞ -category (a quasi-category in the sense of Joyal–Lurie [3]) whose objects are program fragments and whose morphisms are semantic embeddings, with all higher morphisms being invertible witnesses that the embeddings commute with the correctness predicates;
2. J is a Grothendieck topology on the homotopy category $\text{h}\mathcal{S}$, lifted to $\mathcal{S}$ via the nerve construction.

Remark 3.2. For the purposes of this paper the reader may think of $\mathcal{S}$ as an ordinary small category; the ∞ -categorical enhancement is needed only to ensure that the resulting sheaf ∞ -topos (Section 4) satisfies the internal-language theorem at the level of MLTT with univalence [4].

3.1 The Semantic Site of JuGeo

Concretely, the semantic site $\mathcal{S}$ of the JuGeo framework has:

Objects.

Pairs (U, φ_U) where U is a program fragment (module, function, region) and φ_U is its associated correctness proposition.

Morphisms.

Semantic embeddings: maps $f : U \rightarrow V$ that reflect correctness, meaning $\varphi_V \circ f \vdash \varphi_U$.

Covers.

A family $\{f_i : U_i \rightarrow U\}$ covers U if the combined evidence from the U_i suffices to verify U by the sheaf-descent axiom of Paper 3.

The higher morphisms in $\mathcal{S}$ are proof-relevant: a 2-morphism $\alpha : f \Rightarrow g$ is a homotopy between two semantic embeddings, i.e., a verification that the two embeddings produce extensionally equal judgments on all inputs.

4 The ∞ -Topos $\mathrm{Sh}(\mathcal{S})$ as a Model of HoTT

Definition 4.1 (∞ -topos of sheaves). The ∞ -topos of sheaves on $\mathcal{S}$ is

$$\mathrm{Sh}(\mathcal{S}) := \mathrm{PSh}(\mathcal{S})^{\sim J}$$

the full sub- ∞ -category of the presheaf ∞ -category $\mathrm{PSh}(\mathcal{S}) = \mathrm{Fun}(\mathcal{S}^{\mathrm{op}}, \infty\text{-Grpd})$ spanned by those presheaves satisfying the J -sheaf condition (homotopy descent for all covering sieves).

The sheaf condition says: for every covering family $\{f_i : U_i \rightarrow U\}$, the canonical map

$$\mathcal{F}(U) \xrightarrow{\sim} \lim_{\Delta} \mathcal{F}(U_{\bullet})$$

is an equivalence of ∞ -groupoids, where $U_{\bullet}$ is the Čech nerve of the cover. This is precisely the categorical version of the descent condition studied in Paper 3.

Theorem 4.2 (Sheaf Model of MLTT+UA). $\mathrm{Sh}(\mathcal{S})$ carries a model of Martin-Löf type theory with:

1. **Π -types, Σ -types, identity types, and a hierarchy of universes** $\mathcal{U}_0 \subset \mathcal{U}_1 \subset \dots$.
2. **Univalence:** for any two objects $\mathcal{F}, \mathcal{G} \in \mathrm{Sh}(\mathcal{S})$ regarded as types, the map $\mathrm{ua} : (\mathcal{F} \simeq \mathcal{G}) \rightarrow (\mathcal{F} = \mathcal{G})$ is an equivalence in $\mathrm{Sh}(\mathcal{S})$.
3. **Higher inductive types:** every HIT defined by generators and relations has an interpretation in $\mathrm{Sh}(\mathcal{S})$ by the colimit construction of Lurie [3].

Proof sketch. The existence of a model of MLTT with univalence in any ∞ -topos satisfying descent is a theorem of Shulman [4]. The semantic site $\mathcal{S}$ satisfies the required descent condition by construction (Paper 3, Theorem 4.2). The universe hierarchy is constructed using the object classifier $\tilde{\mathcal{U}}$ of $\mathrm{Sh}(\mathcal{S})$, which exists because $\mathrm{Sh}(\mathcal{S})$ is locally cartesian closed and presentable. Univalence follows from the fact that the object classifier carries an identity type whose path space is the type of equivalences, by the universal property of the classifying map. $\square$

4.1 Types as Sheaves and Terms as Sections

Under Theorem 4.2 the internal language of $\mathbf{Sh}(\mathcal{S})$ interprets:

Type-theoretic notion	Sheaf-theoretic counterpart
Type $A : \mathcal{U}$	Sheaf $\mathcal{A} \in \mathbf{Sh}(\mathcal{S})$
Term $a : A$	Global section $a \in \Gamma(\mathcal{S}, \mathcal{A})$
Dependent type $P : A \rightarrow \mathcal{U}$	Fibration $\mathcal{P} \twoheadrightarrow \mathcal{A}$ in $\mathbf{Sh}(\mathcal{S})$
Σ -type $\sum_{a:A} P(a)$	Total space of $\mathcal{P} \rightarrow \mathcal{A}$
Π -type $\prod_{a:A} P(a)$	Space of sections of $\mathcal{P} \rightarrow \mathcal{A}$
Identity type $a =_A b$	Path space fibration in $\mathcal{A}$
Universe $\mathcal{U}$	Object classifier of $\mathbf{Sh}(\mathcal{S})$
Equivalence $A \simeq B$	Equivalence of sheaves

5 Judgment Types and Evidence Bundles

Recall from Paper 2 that a judgment is an 8-tuple $\mathcal{J} = (\mathbf{c}, \varphi, \mathcal{A}, \mathcal{E}, \mathcal{O}, \mathcal{B}, \mathcal{T}, \Pi)$. We now endow this data with a type-theoretic interpretation.

Definition 5.1 (Judgment type). For a judgment template $\mathcal{J}$ over $\mathcal{S}$, the *judgment type* is the sheaf

$$\mathbf{JType}(\mathcal{J}) : \mathcal{S}^{\text{op}} \longrightarrow \infty\text{-Grpd}$$

defined on objects by

$$\mathbf{JType}(\mathcal{J})(U) = \left\{ (\mathcal{E}_U, t_U) \mid \mathcal{E}_U : \mathcal{E}(U, \mathcal{J}), \ t_U \geq \mathcal{T}(\mathcal{J}), \ \mathcal{B}(U, \mathcal{J}) = \emptyset \right\},$$

the ∞ -groupoid of evidence bundles for $\mathcal{J}$ at U that satisfy the trust floor $\mathcal{T}(\mathcal{J})$ and carry no outstanding obstructions. Restriction maps are induced by the semantic embedding functoriality of evidence (Paper 2, Definition 3.4).

Proposition 5.2 (Judgment types are sheaves). *For every judgment $\mathcal{J}$, $\mathbf{JType}(\mathcal{J})$ satisfies the $\mathcal{J}$ -sheaf condition, i.e., it belongs to $\mathbf{Sh}(\mathcal{S})$.*

Proof. Let $\{f_i : U_i \rightarrow U\}$ be a covering family. By Paper 3, Theorem 4.2, any compatible family of evidence bundles $(\mathcal{E}_{U_i})_i$ satisfying the trust and obstruction conditions Čech-descends to a unique evidence bundle $\mathcal{E}_U$ over U . The obstruction condition $\mathcal{B}(U, \mathcal{J}) = \emptyset$ follows from the vanishing of the first Čech cohomology class (Paper 3, Corollary 4.5). Thus the sheaf condition holds. $\square$

Definition 5.3 (Verified program). A program P is *verified* with respect to judgment $\mathcal{J}$ if it corresponds to a global section

$$\sigma_P \in \Gamma(\mathcal{S}, \mathbf{JType}(\mathcal{J})) = \mathbf{JType}(\mathcal{J})(\text{terminal}).$$

A global section assigns to every program fragment U an evidence bundle for $\mathcal{J}|_U$ that is compatible with all restrictions.

This recovers the original JUGE0 definition of verification while embedding it in the richer type-theoretic universe $\mathbf{Sh}(\mathcal{S})$. The type $\Gamma(\mathcal{S}, \mathbf{JType}(\mathcal{J}))$ is an object of $\mathbf{Sh}(\mathcal{S})$, and the *verified programs* form its terms.

6 Univalence for Program Semantics

6.1 The Type of Programs

Definition 6.1 (Type of programs). The *type of programs* is the sheaf

$$\mathbf{Prog} := \{ (A, \sigma_A) \mid A : \mathbf{ProgramFragment}, \sigma_A \in \Gamma(\mathcal{S}, \mathbf{JType}(\mathcal{J}_A)) \} \in \mathbf{Sh}(\mathcal{S}),$$

where $\mathcal{J}_A$ denotes the judgment template induced by the specification of A . Thus $\mathbf{Prog}$ is the type of *verified* programs.

6.2 Behavioral Equivalence as Equivalence of Types

Definition 6.2 (Program equivalence). Two programs $A, B : \mathbf{Prog}$ are *behaviorally equivalent*, written $A \simeq_{\mathbf{Prog}} B$, if there exists an equivalence of evidence-bundle types

$$e : \Gamma(\mathcal{S}, \mathbf{JType}(\mathcal{J}_A)) \simeq \Gamma(\mathcal{S}, \mathbf{JType}(\mathcal{J}_B))$$

that commutes with the semantic embedding functors. Concretely, e is an isomorphism of verification witnesses that maps every valid evidence bundle for A to a valid evidence bundle for B and vice versa.

Theorem 6.3 (Univalence for programs). *In $\mathbf{Sh}(\mathcal{S})$, behavioral equivalence of programs corresponds bijectively to paths in $\mathbf{Prog}$:*

$$(A \simeq_{\mathbf{Prog}} B) \simeq (A =_{\mathbf{Prog}} B).$$

Under this correspondence, the reflexivity path $\mathbf{refl}_A$ corresponds to the identity equivalence.

Proof. This is the specialisation of the univalence axiom (Theorem 4.2, item 2) to the sub-universe spanned by $\mathbf{Prog} \subset \mathcal{U}$. An equivalence $e : A \simeq_{\mathbf{Prog}} B$ consists of a quasi-inverse pair of section maps; the univalence map $\mathbf{ua}(e)$ produces the corresponding path $p : A =_{\mathbf{Prog}} B$. The map $\mathbf{ua}$ is an equivalence by Theorem 4.2, so $(A \simeq_{\mathbf{Prog}} B) \simeq (A =_{\mathbf{Prog}} B)$. $\square$

Corollary 6.4. *Every verified refactoring $r : A \rightsquigarrow B$ (a morphism in $\mathcal{S}$ that transports all judgment-sheaf sections from A to B with trust level $\geq \mathbf{PROOF}$) induces a path $\bar{r}(A) =_{\mathbf{Prog}} \bar{r}(B)$ in the type of programs.*

Proof. A verified refactoring r induces an evidence-bundle equivalence $e_r : A \simeq_{\mathbf{Prog}} B$ by functoriality of $\mathbf{JType}$ along r . Apply Theorem 6.3. $\square$

7 Higher Inductive Types for Program Transformations

Beyond single refactorings, real-world program evolution involves *sequences* and *reversals* of transformations. Higher inductive types let us encode this structure directly.

7.1 The Refactoring Interval

Definition 7.1 (Refactoring interval). Given programs $A, B : \mathbf{Prog}$ and a verified refactoring $r : A \rightsquigarrow B$, the *refactoring interval* $\mathbb{I}_{A,B,r}$ is the HIT with

- point constructors $\mathbf{src} : A$ and $\mathbf{tgt} : B$;
- path constructor $\mathbf{seg}_r : \mathbf{src} =_{\mathbf{Prog}} \mathbf{tgt}$.

The interval type allows proofs about intermediate states: any type-theoretic property P that holds at A and is preserved by the refactoring step is automatically available along the entire path seg_r .

7.2 The Space of Refactorings

Let $\text{Refact}(A, B)$ denote the type of all verified refactorings from A to B . We define:

$$\text{Refact}(A, B) := \{ r : A \rightsquigarrow B \mid e_r : A \simeq_{\text{Prog}} B, \mathcal{T}(r) \geq \text{PROOF} \}$$

Proposition 7.2 (Refactoring type is a set). *$\text{Refact}(A, B)$ is a set (0-truncated type) in $\text{Sh}(\mathcal{S})$: any two refactorings $r, s : \text{Refact}(A, B)$ that induce the same equivalence $e_r = e_s$ are propositionally equal.*

Proof. Two refactorings that induce the same equivalence of evidence bundles produce the same path in Prog by Theorem 6.3. Since paths in Prog between fixed endpoints form a groupoid (being the path space of a sheaf), and Refact is defined by a subtype of verified morphisms in $\mathcal{S}$ (a 1-category), the 0-truncation condition holds. $\square$

7.3 Pushout Programs

A common refactoring pattern is *merging* two programs that share a common ancestor. The HIT encoding uses a homotopy pushout.

Definition 7.3 (Program pushout). Given programs $A, B, C : \text{Prog}$ and verified refactorings $l : C \rightsquigarrow A$, $r : C \rightsquigarrow B$, the *pushout program* $A \sqcup_C B$ is the HIT with

- point constructors $\text{inl} : A \rightarrow A \sqcup_C B$ and $\text{inr} : B \rightarrow A \sqcup_C B$;
- path constructor $\text{glue}(c) : \text{inl}(l(c)) =_{A \sqcup_C B} \text{inr}(r(c))$ for each $c : C$.

The pushout program provides the universal merge point: any proof that holds for both A and B and is compatible at C extends to $A \sqcup_C B$ by the universal property of the homotopy pushout.

8 Proof Transfer via Transport

We now formalise the main application: automated proof transfer along verified refactorings.

Definition 8.1 (Program property). A *program property* is a dependent type $P : \text{Prog} \rightarrow \mathcal{U}$ in the internal language of $\text{Sh}(\mathcal{S})$. Typical examples include:

- $\text{Terminates}(A)$ — program A terminates on all inputs in its specification domain;
- $\text{SatisfiesSpec}(A, \varphi)$ — A satisfies logical specification φ ;
- $\text{BoundedComplexity}(A, f)$ — A 's runtime is bounded by f on all inputs.

Theorem 8.2 (Proof transfer). *Let $P : \text{Prog} \rightarrow \mathcal{U}$ be a program property, let $A, B : \text{Prog}$, and let $r : \text{Refact}(A, B)$ be a verified refactoring with $\mathcal{T}(r) \geq \text{PROOF}$. Then there is a canonical function*

$$\text{transfer}(P, r) : P(A) \longrightarrow P(B)$$

defined by transport along the path $\text{ua}(e_r) : A =_{\text{Prog}} B$.

Proof. By Corollary 6.4, r induces a path $p_r := \text{ua}(e_r) : A =_{\text{Prog}} B$. By Definition 2.2, transport gives $\text{transport}(P, p_r) : P(A) \rightarrow P(B)$. Set $\text{transfer}(P, r) := \text{transport}(P, p_r)$. $\square$

Remark 8.3. Theorem 8.2 is completely constructive: given a machine-verified refactoring r (trust level **PROOF** in the JuGEO trust algebra) and a proof $h : P(A)$, the system automatically constructs $\text{transfer}(P, r)(h) : P(B)$ by applying the transport eliminator. No additional tactic scripting or lemma writing is required.

Corollary 8.4 (Proof transfer along refactoring chains). *If $r_1 : \text{Refact}(A, B)$ and $r_2 : \text{Refact}(B, C)$, then the composed refactoring $r_2 \circ r_1$ induces a transfer*

$$\text{transfer}(P, r_2 \circ r_1) : P(A) \rightarrow P(C)$$

equal to $\text{transfer}(P, r_2) \circ \text{transfer}(P, r_1)$.

Proof. Path concatenation is functorial under transport: $\text{transport}(P, p \cdot q) = \text{transport}(P, q) \circ \text{transport}(P, p)$ for any paths p, q . Apply this to the concatenated path $p_{r_1} \cdot p_{r_2}$. $\square$

8.1 Transfer of JuGeo Judgment Certificates

Concretely, in the JuGEO framework, proof transfer operates as follows. Suppose program A has been fully verified:

- The verification engine has produced a global section $\sigma_A \in \Gamma(\mathcal{S}, \text{JType}(\mathcal{J}_A))$ at trust level **PROOF**.
- A developer performs a refactoring $r : A \rightsquigarrow B$ that the **TRANSPORT** move certifies at trust level **PROOF**.

Then $\text{transfer}(\text{JType}(\mathcal{J}_-), r)(\sigma_A)$ yields $\sigma_B \in \Gamma(\mathcal{S}, \text{JType}(\mathcal{J}_B))$, a full verification certificate for B , without re-running the verification engine.

Example 8.5 (Sorting algorithm transfer). Let $A = \text{quicksort}$ and let $r : A \rightsquigarrow B$ be a refactoring to $B = \text{mergesort}$ that has been formally verified to preserve the specification $\varphi =$ “output is a sorted permutation of input”. Then $\text{transfer}(\text{SatisfiesSpec}(-, \varphi), r)$ transports the quicksort correctness certificate to a mergesort certificate automatically. The refactoring trust level must be $\geq \text{PROOF}$ for the transfer to carry proof-level guarantees.

9 The Fundamental Group of Programs

The *loop space* of **Prog** at a basepoint $A : \text{Prog}$ is the type of self-refactoring loops:

$$\text{Loop}(\text{Prog}, A) := A =_{\text{Prog}} A.$$

This is the type of all paths in **Prog** that start and end at A , i.e., all verified refactorings of A that return A to itself (up to behavioral equivalence).

Definition 9.1 (Fundamental group). The *fundamental group* of **Prog** at A is the 0-truncation:

$$\pi_1(\text{Prog}, A) := \|\text{Loop}(\text{Prog}, A)\|_0,$$

the set of path-connected components of the loop space, equipped with the group structure inherited from path concatenation.

Theorem 9.2 (Classification of refactoring loops). $\pi_1(\text{Prog}, A)$ is isomorphic to the group

$$\text{RefactLoop}(A) := \text{Refact}(A, A)/\sim$$

of verified self-refactoring loops modulo homotopy (i.e., modulo 2-paths in $\text{Sh}(\mathcal{S})$). The group multiplication is composition of refactorings; the identity is the trivial refactoring id_A ; inverses are refactoring reversals.

Proof. By Theorem 6.3, $\text{Refact}(A, A) \simeq \text{Loop}(\text{Prog}, A)$. Taking 0-truncations on both sides and using Proposition 7.2 (which ensures Refact is already a set), we obtain $\text{RefactLoop}(A) \simeq \|\text{Loop}(\text{Prog}, A)\|_0 = \pi_1(\text{Prog}, A)$. $\square$

Corollary 9.3 (Invariant preservation). *A program property $P : \text{Prog} \rightarrow \text{Prop}$ is a refactoring invariant of A — preserved by every verified self-refactoring of A — if and only if $P(A)$ is fixed by the action of $\pi_1(\text{Prog}, A)$ on the fiber $P(A)$.*

Proof. The action of $\pi_1(\text{Prog}, A)$ on $P(A)$ is defined by $[\gamma] \cdot h := \text{transport}(P, \gamma, h)$ for $[\gamma] \in \pi_1(\text{Prog}, A)$ and $h : P(A)$. Since $P(A)$ is a proposition (element of Prop), the action is trivial, meaning $\text{transport}(P, \gamma, h) = h$ for all $\gamma : \text{Loop}(\text{Prog}, A)$. Thus every propositional property is automatically a refactoring invariant. $\square$

9.1 Example: The Loop Group of a Pure Function

Consider a pure function $f : \mathbb{Z}^n \rightarrow \mathbb{Z}$ verified against a polynomial specification φ_f . The group $\pi_1(\text{Prog}, f)$ consists of verified refactoring loops of f : loop elements include renaming of local variables (identity in homotopy), reordering of commutative operations (nullhomotopic for associative operations), and algebraic simplifications provably equivalent to the identity transformation. Non-trivial elements arise from verified optimisations that change runtime behaviour while preserving the extension; these correspond to non-trivial 2-cells in $\mathcal{S}$.

10 Implementation in JuGeo

The theoretical development above is accompanied by a LEAN 4.4 formalisation in `Paper20_HomotopyTypeTheory.1`. The formalisation works within the JU GEO Lean library and proves:

1. **Transport along paths** (Theorem 8.2): implemented using Lean’s built-in `Eq.subst` and the substitution eliminator. Verified without `sorry`.
2. **Function extensionality for programs**: if two verified programs agree on all inputs (i.e., their behaviour functions are pointwise equal), then they are propositionally equal. Proved using Lean’s `funext` axiom.
3. **Equivalence implies equality** (simplified): if two program-type propositions are logically equivalent (provable iff), they are propositionally equal by `propext`.
4. **Proof-transfer pipeline**: a full pipeline

$$(P(A), r : A \simeq_{\text{Prog}} B) \implies P(B)$$

implemented as a sequence of type-theoretic steps.

5. **Refactoring loop group:** a concrete group structure on self-paths, with the group laws verified from the Lean path algebra.

The formalisation avoids the full ∞ -topos machinery by working with a simplified model: programs are Lean types, paths are propositional equalities, and equivalences are `Equiv` records. Within this model, all stated theorems have complete proofs.

10.1 Univalence in the Lean Model

Lean 4 itself is based on Calculus of Constructions with UIP (uniqueness of identity proofs), which makes the full univalence axiom inconsistent when added naively. Our companion file handles this by:

- Stating the univalence axiom as a `noncomputable axiom` (consistent with Lean’s foundations when restricted to types that live in a univalent fragment);
- Proving all downstream results from this axiom without `sorry`, so that the logical dependency is explicit.

This is the standard approach used in the Lean community’s `Mathlib.Logic.Equiv.Basic` and the `Cubical` library.

11 The Čech–de Rham Spectral Sequence Revisited

Paper 3 computed the first Čech cohomology group $H^1(\mathcal{S}, \mathbf{JType}(\mathcal{J}))$ as an obstruction to global verification. The HoTT interpretation gives a conceptually cleaner picture.

Proposition 11.1. *In the internal language of $\mathbf{Sh}(\mathcal{S})$:*

$$\begin{aligned} H^0(\mathcal{S}, \mathbf{JType}(\mathcal{J})) &= \Gamma(\mathcal{S}, \mathbf{JType}(\mathcal{J})) \quad (\text{verified programs}), \\ H^1(\mathcal{S}, \mathbf{JType}(\mathcal{J})) &= \pi_0(\Omega \mathbf{JType}(\mathcal{J})) \quad (\text{connected components of the loop space}), \\ H^k(\mathcal{S}, \mathbf{JType}(\mathcal{J})) &= \pi_0(\Omega^k \mathbf{JType}(\mathcal{J})) \quad (\text{higher homotopy groups of the judgment type}). \end{aligned}$$

Proof sketch. This follows from the Dold–Thom theorem applied to the sheaf $\mathbf{JType}(\mathcal{J})$ viewed as a spectrum in $\mathbf{Sh}(\mathcal{S})$. The identification of H^1 with the loop space components recovers the classical Grothendieck obstruction theory in the language of ∞ -groupoids. $\square$

Proposition 11.1 implies that the obstructions computed by JuGEO’s Čech descent (Paper 3) are exactly the non-trivial connected components of the judgment-type loop space. A verification failure corresponds to a program that lies in a connected component of $\mathbf{JType}(\mathcal{J})$ without a global section, i.e., a program whose judgment loop is non-contractible.

12 Related Work

HoTT and programming languages. The original HoTT Book [1] established the foundational programme of interpreting type theory in ∞ -groupoids. Licata and Shulman [6] and Awodey [5] developed the connection to algebraic topology. Shulman [4] proved that every Grothendieck ∞ -topos provides a model of MLTT + UA, which is the model theorem we specialise in Theorem 4.2.

Program verification via type theory. `LEAN 4`, `COQ`, and `F*` provide dependent-type verification, but do not use the geometric structure of $\mathbf{Sh}(\mathcal{S})$ and do not provide automatic proof transfer. The closest prior work is the parametricity-based transfer used in Lean’s `Equiv.transfer` and Coq’s `Program.transfer`, which transfer proofs along isomorphisms but require manual specification of the isomorphism; our work automates this via the semantic-site structure.

Categorical semantics of programs. The use of sheaves and descent for program verification is developed in Papers 1–10 of this series. Jacobs [7] studies categorical logic and fibrations for dependent type theories, which underlies the fiber interpretation of Sections 4–5.

Homotopy groups of program spaces. The idea of studying programs via their homotopy type appears in Finster–Mimram [8], who compute π_1 of the space of finitary symmetric monoidal categories using rewriting systems. Our $\pi_1(\mathbf{Prog}, A)$ is the analog for verified programs on a semantic site.

13 Conclusion

We have established that the ∞ -topos $\mathbf{Sh}(\mathcal{S})$ of sheaves on the JU GEO semantic site carries a model of Martin–Löf type theory with univalence. Within this model:

- Judgments are types whose terms are evidence bundles;
- Verified programs are global sections of judgment-type sheaves;
- Behavioral equivalence of programs corresponds to paths in the type `Prog`, and univalence promotes equivalences to equalities;
- Transport along paths implements automatic proof transfer across verified refactorings;
- The fundamental group $\pi_1(\mathbf{Prog}, A)$ classifies refactoring loops up to homotopy, and propositional properties are automatically fixed by this action.

The companion `LEAN 4 4` development formalises the central theorems without `sorry`, grounding the abstract constructions in a concrete mechanically verified development.

Future work includes: (i) computing $\pi_1(\mathbf{Prog}, A)$ for concrete program families; (ii) extending the Čech spectral sequence interpretation to higher cohomology groups; (iii) implementing the proof-transfer pipeline in the JU GEO Python framework so that `TRANSPORT` moves automatically generate transferred certificates.

Acknowledgements

The author thanks the Lean4 Mathlib community for foundational infrastructure on which the companion Lean file builds.

References

- [1] The Univalent Foundations Program. *Homotopy Type Theory: Univalent Foundations of Mathematics*. Institute for Advanced Study, 2013. <https://homotopytypetheory.org/book/>

- [2] V. Voevodsky. Univalent foundations of mathematics. In *Logic, Language, Information, and Computation*, 2010.
- [3] J. Lurie. *Higher Topos Theory*. Annals of Mathematics Studies, vol. 170. Princeton University Press, 2009.
- [4] M. Shulman. All $(\infty, 1)$ -toposes have strict univalent universes. *arXiv:1904.07004*, 2019.
- [5] S. Awodey. Type theory and homotopy. In *Epistemology versus Ontology*, pp. 183–201. Springer, 2012.
- [6] D. Licata and M. Shulman. Calculating the fundamental group of the circle in homotopy type theory. In *LICS 2013*, pp. 223–232. IEEE, 2013.
- [7] B. Jacobs. *Categorical Logic and Type Theory*. Studies in Logic, vol. 141. Elsevier, 1999.
- [8] E. Finster and S. Mimram. A type-theoretical definition of weak ω -categories. In *LICS 2017*. IEEE, 2017.